DATS: Degree-Aware Tiled Search for Fast Max-Cut Local Search on GPUs

Vedant Tamhanekar
Veermata Jijabai Technological Institute (VJTI)
Mumbai, India

Siddhant Salunke
Veermata Jijabai Technological Institute (VJTI)
Mumbai, India

***Abstract*—Local search methods like tabu search are among the best heuristics for the Max-Cut problem, and earlier work has shown that running many searches in parallel on a GPU gives large speedups. Most of these GPU codes give one thread to each vertex and are tested on G-set graphs, where all vertices have almost the same degree. On real power-law graphs this design has two problems. First, when a hub vertex is flipped, one thread has to update thousands of neighbours while the rest of the block waits. Second, and less discussed, every step scans all n vertices to find the best move even though a flip changes only the gains of the flipped vertex and its neighbours. In this paper we present DATS (Degree-Aware Tiled Search), a GPU tabu search for Max-Cut that addresses both. DATS keeps the best move of every 32-vertex tile in shared memory and after each flip recomputes only the tiles that changed, using one warp per tile. The neighbour update is given to one warp or to the whole block depending on the degree of the flipped vertex, and a BFS relabelling keeps neighbours in the same tiles. The tiled selection is exact, so DATS makes exactly the same moves as a full scan. On four SNAP graphs, a 100k-vertex power-law graph and two sparse G-set graphs, tiled selection reads 13–28× fewer entries per step and alone gives 4.6–8.0× speedup on a single CPU core. Together with OpenMP on a 10-core CPU it reaches up to 37× over the serial full-scan search. On an NVIDIA Tesla T4, DATS is 9.1–15.9× faster than a one-thread-per-vertex GPU baseline on the graphs with more than 15k vertices, while degree-aware updates alone give almost no gain there. On small G-set graphs the tile bookkeeping costs more than it saves, and a plain full scan is faster.**

***Index Terms*—Max-Cut, tabu search, GPU, CUDA, load balancing, power-law graphs, Ising model, incremental selection.**

I. Introduction

In the Max-Cut problem we are given a graph and we have to split its vertices into two groups so that the total weight of the edges going between the groups is as large as possible. It is one of Karp's original NP-complete problems [1], and it shows up in VLSI design, statistical physics and network analysis. Max-Cut is also the same as finding the ground state of an Ising spin glass [2], which is why it is often used to test Ising machines and quantum annealers. The best approximation algorithm, by Goemans and Williamson [3], is based on semidefinite programming and is too slow for big graphs, so in practice heuristics such as Breakout Local Search (BLS) [4] are used.

Local search is a good fit for GPUs because many independent searches can be run at the same time. Cook et al. [5] solved Max-Cut with GPU Ising annealing and reported speedups of about 2000× over a CPU on their biggest graphs, and Yasudo et al. [6] ran thousands of searches in parallel on multiple GPUs. However, in [5] each thread looks after one vertex, and the authors point out that the threads are not load balanced when vertices have different degrees, but leave this open. Most of these works are also tested on the G-set [7], whose graphs have nearly uniform degrees. Real graphs like e-mail, social or co-author networks follow a power-law degree distribution [24] (Fig. 7): a few hubs have thousands of neighbours while most vertices have less than ten.

Our first idea was simply to share the neighbour update of a hub among many threads (Fig. 1). While building this we noticed something more important. In a block that runs one tabu search, every step does two things: (1) find the best allowed move, which scans all n gains, and (2) apply it, which touches only $\deg(v)$ neighbours. On our graphs the average degree is between 4 and 48, while n is up to 100,000, so the scan costs hundreds of times more than the update. Balancing the update alone therefore cannot give much. The scan is also mostly wasted work, because only a few gains changed since the last step.

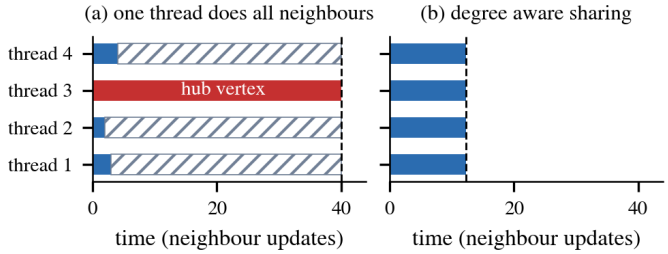


Fig. 1. Load imbalance when a hub vertex is flipped. (a) With one thread per vertex, the thread owning the hub does all updates while the others wait. (b) When the work is shared, all threads finish at about the same time.

Based on this, the main contributions of this paper are:

- **Tiled incremental selection.** Vertices are grouped into tiles of 32. Each tile keeps its best non-tabu move, its best tabu move (for aspiration) and the step when its next tabu vertex becomes free. After a flip only dirty tiles are recomputed, one warp per tile, and the best move is found from $n/32$ summaries. We prove that this gives exactly the same move as a full scan.
- **Degree-aware neighbour updates.** The gain update of the flipped vertex is done by one warp (low and medium degree) or the whole thread block (hubs). We show that this matters more after the selection is made cheap.
- **BFS relabelling** of vertices so that neighbours fall in the same tiles, which reduces the number of dirty tiles when the input ids are in a bad order.
- An evaluation on G-set, SNAP and synthetic power-law graphs with a CPU serial, an OpenMP and four GPU variants that all follow the *identical* search path, so speed can be compared fairly and correctness can be checked exactly.

The rest of the paper is organised as follows. Section II covers background on Max-Cut, GPUs and related work. Section III analyses the cost of one search step with a small example. Section IV explains DATS and proves that the tiled selection is exact. Section V gives the experimental setup and Section VI the results, including scaling, ablation and quality studies. Section VII discusses limitations and Section VIII concludes. The appendix describes how to reproduce all results.

II. Background And Related Work

A. Max-Cut and Local Search

Let $G = (V, E)$ be a graph with n vertices, m edges and weights w_{ij}. A solution is a vector $x \in \{0,1\}^n$ and its cut value is

$$\mathrm{cut}(x) = \sum_{(i,j)\in E} w_{ij}\,[x_i \neq x_j] \tag{1}$$

Local search keeps for every vertex its *gain*, the change of the cut if v is moved to the other side:

$$g_v = \sum_{u \in N(v)} w_{uv}\,(1 - 2\,[x_u \neq x_v]) \tag{2}$$

When v is flipped, its own gain becomes $-g_v$ and every neighbour u is updated as

$$g_u \leftarrow g_u - 2\,w_{uv}\,\sigma_{uv}, \quad \sigma_{uv} = +1 \text{ if } x_u = x_v \text{ else } -1 \tag{3}$$

where σ_{uv} is taken before the flip. So the work of a flip is proportional to $\deg(v)$. Tabu search [8] always makes the best move, even if it is negative, and forbids moving a vertex back for a few steps (its *tenure*). A tabu move is still allowed if it gives a new best cut (aspiration). BLS [4] adds a perturbation phase: when no improvement is found for a number of steps, some random moves are made, and the number of moves grows if the search keeps falling back to the same local optimum. We use these rules in all our versions.

B. Other Max-Cut Solvers and Ising Machines

Besides BLS, many other heuristics have been proposed for Max-Cut. Burer et al. [16] use a rank-two relaxation of the semidefinite program, which is much faster than solving the full relaxation and was for a long time one of the strongest methods on the G-set. Kochenberger et al. [17] show that a tabu search on the QUBO form can handle Max-Cut instances with up to 10,000 vertices, and Wu and Hao [26] combine tabu search with a memetic algorithm. All of these are sequential CPU methods. Their core loop is the same as ours: keep the gains, pick the best move, update the neighbours, so the ideas of this paper can also be used inside them.

Because Max-Cut is the same as finding the ground state of an Ising model [2], it is also a standard benchmark for special hardware. Examples are the coherent Ising machine built with optical pulses [18], the CMOS annealing chip of Yamaoka et al. [20], and simulated bifurcation [19], which runs on GPUs and FPGAs. These machines are fast but need special hardware or update all spins together. Our work stays with ordinary GPUs and an exact tabu search, and focuses on making each step cheaper.

C. GPU Methods for Max-Cut and QUBO

Cook et al. [5] map Max-Cut to an Ising model and run simulated annealing on the GPU with one thread per spin. Yasudo et al. propose Adaptive Bulk Search [6], where each CUDA block runs its own local search on a QUBO problem stored as a dense $n \times n$ matrix, guided by a genetic algorithm on the host. Because of the dense matrix the memory grows as n^2. Their later work [9] stores the QUBO as a sparse graph to scale to bigger instances. We also use one block per search and a sparse graph, so these are not our contribution. As far as we know none of these works changes the number of threads by vertex degree or avoids the full scan for the best move.

D. GPU Architecture Basics

For readers less familiar with CUDA [25], we briefly describe the parts of the GPU that matter for our design (Fig. 2). A kernel is launched as a grid of thread blocks. Each block runs on one streaming multiprocessor (SM) and its threads are grouped into warps of 32 threads that execute the same instruction together. The threads of a block can share data through *shared memory*, which is small (up to 64 KB per block on the T4) but much faster than the global memory used by all blocks. Threads in one block can wait for each other with `__syncthreads()`, and the 32 lanes of a warp can exchange registers directly with shuffle instructions such as `__shfl_down_sync`, which is the usual way to build fast reductions [29].

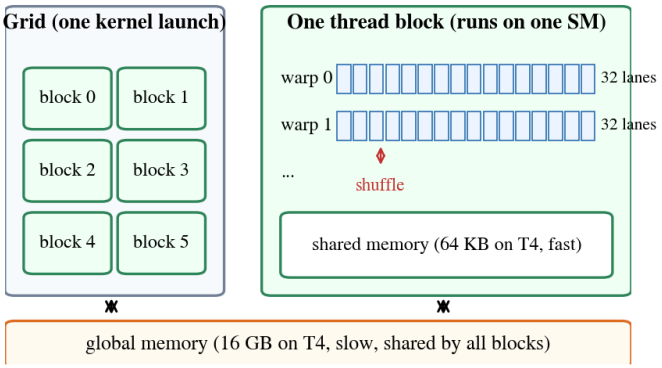


Fig. 2. The CUDA execution model used in this paper: a grid of independent blocks, warps of 32 lanes inside a block, fast per-block shared memory and slower global memory.

Two points follow from this. First, if one thread of a block has much more work than the others, the whole block waits for it at the next synchronisation, which is the load imbalance of Fig. 1. Second, warps are the natural unit of work: 32 values can be reduced inside a warp with five shuffle steps and without touching shared memory. Both points shape our design. A tile has exactly 32 vertices so that one warp can refresh it, and the update of a vertex is given to a warp or to the whole block depending on its degree.

E. Load Balancing and Incremental Gain Structures

Degree imbalance is a well known problem in GPU graph analytics. Merrill et al. [10] process the neighbour list of a vertex with a thread, a warp or the whole block depending on its size, and Gunrock [11] uses similar strategies. Our degree-aware update is an adaptation of this idea to local search, where only one vertex is expanded per step. For finding the best gain fast on a CPU, the Fiduccia–Mattheyses heuristic [12] keeps gain buckets with linked lists. Bucket lists need pointer updates that are serial and do not map well to warps, and they do not handle the tabu and aspiration rules directly. Our tile summaries are a flat, warp-friendly alternative. Hong et al. [21] proposed virtual warp-centric processing, where a group of lanes works on one vertex, which is close to our medium-degree class.

Vertex reordering to improve locality is also standard. Cuthill–McKee [13] reduces the bandwidth of sparse matrices, and newer orderings like Gorder [22] and Rabbit Order [23] place vertices that share neighbours close together to improve cache use in graph analytics. We use a simple BFS version of this idea for a new goal, which is to reduce the number of dirty tiles after a flip. Better orderings could be plugged in directly.

III. Cost Of One Search Step

Table I lists the symbols used in the rest of the paper.

TABLE I
NOTATION

Symbol	Meaning
n, m	number of vertices and edges
x_v	side of vertex v (0 or 1)
g_v	gain of v, change of the cut if v is flipped
$N(v)$, $\deg(v)$	neighbours and degree of v
P	threads per block
T	tile size (32 on the GPU)
F_t, T_t	best non-tabu and best tabu move key of tile t
E_t	first step at which a tabu vertex of tile t becomes free
$D(v)$	number of different tiles in $N(v) \cup \{v\}$
$K(v)$	64-bit move key of v, see (4)

A. A Small Example

Fig. 3 shows one step on a small graph with six vertices and nine unit-weight edges. In (a) the cut is 5 and the gains are computed with (2). The largest gain is +2 at vertex B, so a tabu search flips B. After the flip (b) the cut is 7, the gain of B becomes -2, and only the gains of its neighbours A, C, D and E change, each by ±2 as in (3). The gain of every other vertex stays the same. Still, a full-scan search looks at all six gains again in the next step to find the best move. On a graph with 100,000 vertices and average degree 8, one flip changes about 9 gains, but the next step reads all 100,000 of them. This is the waste we remove in Section IV.

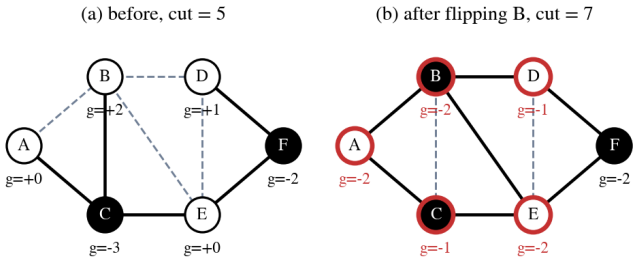


Fig. 3. One search step on a small graph. Black and white show the two sides, solid edges are cut and dashed edges are not. Flipping B (the largest gain) changes only the gains of B and its neighbours (red).

B. Where the Time Goes

Consider one tabu search run by a block of P threads. In the usual design ("full scan"), a step reads all n gains and tabu values to find the best allowed move, so the selection costs n/P per thread plus a reduction. The update then costs $\deg(v)$ if one thread does it (naive), or about $\deg(v)/32$ or $\deg(v)/P$ with degree-aware sharing. Table II shows that on our power-law graphs a step scans 7,000 to 100,000 entries while the average degree is about 10. Even the largest hub in powerlaw-100k (degree 16,759) is smaller than n. Also, we measured that hubs (degree above 1024) are selected in only 0.01–0.21% of steps. We think this is because a hub has a large gain only for a short time, and after it is flipped it stays tabu for many steps. This tells us that the selection is the main cost and has to be fixed first, and the hub update becomes important only after that. This is the motivation behind combining the two techniques.

TABLE II
GRAPHS USED IN THE EXPERIMENTS

Graph	Type	n	m	avg deg	max deg	hubs
G1	random	800	19,176	47.9	67	0
G14	planar	800	4,694	11.7	132	0
G22	random	2,000	19,990	20.0	37	0
G43	random	1,000	9,990	20.0	36	0
G48	toroidal	3,000	6,000	4.0	4	0
G55	random	5,000	12,498	5.0	15	0
wiki-Vote	SNAP	7,115	100,762	28.3	1,065	1
email-Enron	SNAP	36,692	183,831	10.0	1,383	9
ca-AstroPh	SNAP	18,772	198,050	21.1	504	0
soc-Epinions1	SNAP	75,879	405,740	10.7	3,044	12
powerlaw-100k	synthetic, γ=2.1	100,000	355,597	7.1	16,759	–

IV. Proposed Method: DATS

Fig. 4 shows the overall design. The host reads the graph, builds the CSR arrays, relabels the vertices and computes the degree class of each vertex. The graph is copied once to global memory and shared by all blocks. Every block runs one independent tabu search and keeps its search state in shared memory.

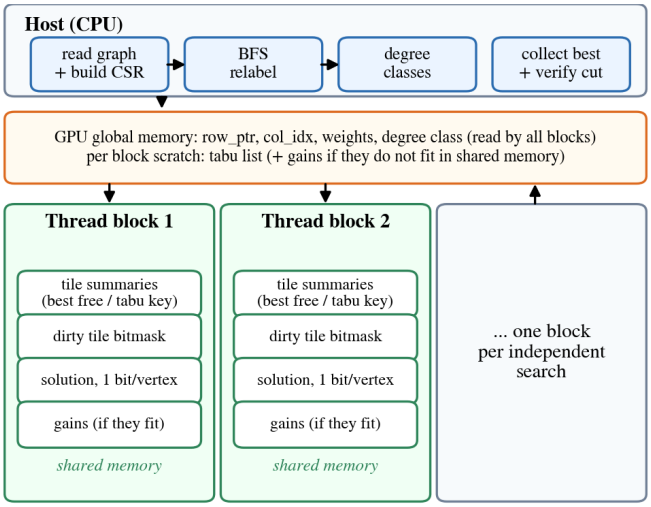


Fig. 4. Mapping of DATS on the GPU. All blocks read the same CSR graph from global memory, each block keeps its own solution, tile summaries and (if they fit) gains in shared memory.

A. Move Keys

To compare moves with a single integer comparison, we pack each candidate move into a 64-bit key:

$$K(v) = ((g_v \oplus 2^{31}) \ll 32) \mid ((v \cdot \phi \bmod 2^{32}) \oplus s) \qquad (4)$$

The upper half is the gain with its sign bit flipped, so that a larger gain gives a larger unsigned number. The lower half is a tie-break value: φ is the odd constant 0x9E3779B1 and s is a random salt. Because φ is odd it has an inverse modulo 2^{32}, so the vertex can be recovered from the key. The salt is changed after every perturbation, so ties are broken in a different random order in each phase. With this encoding, "best move" is just the maximum key, which works directly with warp shuffle reductions.

B. Tiled Incremental Selection

The vertices are divided into tiles of 32 consecutive ids, one per warp lane. For each tile t the block stores three values: F_t, the maximum key among the non-tabu vertices; T_t, the maximum key among the tabu vertices; and E_t, the smallest step at which one of its tabu vertices becomes free. A tile can only become wrong in two ways: a gain or tabu value inside it changes (only after a flip of v or a neighbour of v), or the step reaches E_t. So during the neighbour update each thread also sets the bit of the neighbour's tile in a dirty bitmask using `atomicOr`. After the flip, the warps go over the bitmask and every dirty tile is recomputed by one warp, where lane i reads vertex $32t+i$ and three shuffle reductions give F_t, T_t and E_t (Fig. 5).

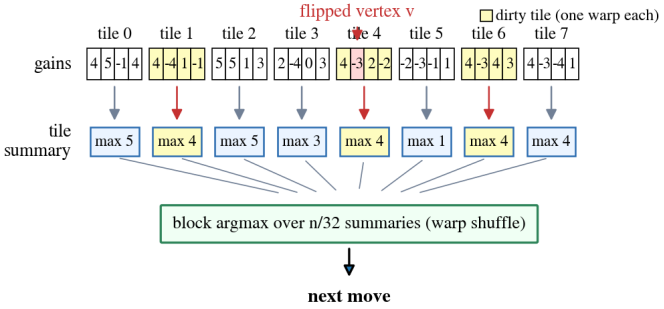


Fig. 5. Tiled incremental selection. After flipping v, only the tiles that hold v or its neighbours are recomputed. The next move is the maximum over the tile summaries.

To choose a move, the threads go over the tile summaries. If $E_t \le$ step, the tile is refreshed first (this is rare, at most one vertex becomes free per step on average). The candidate of a tile is F_t, or T_t if T_t passes the aspiration test and is larger. A block reduction gives the global maximum.

Exactness. We now show that the tiled selection always returns the same move as a full scan. Let $A(s)$ be the set of vertices allowed at step s, i.e. the non-tabu vertices plus the tabu vertices that pass the aspiration test cut + g_v > best.

Lemma 1. *Let the summary of tile t be computed at step s_0. If no vertex of t has its gain or tabu value changed and the salt is not changed, then for every step s with $s_0 \le s < E_t$ the values F_t and T_t are the largest keys among the non-tabu and the tabu vertices of t at step s.*

Proof. A vertex is tabu at step s if its tabu value is larger than s. A vertex that was free at s_0 stays free, since its tabu value did not change. A vertex that was tabu at s_0 has a tabu value of at least $E_t > s$, so it is still tabu. So the two groups are the same as at s_0. The keys depend only on the gain, the vertex id and the salt, which did not change, so the maxima are the same. ∎

Theorem 1. *At every step, the move chosen by the tiled selection is the vertex with the largest key in $A(s)$, i.e. the same move as a full scan.*

Proof. A gain or tabu value changes only for the flipped vertex and its neighbours, and all their tiles are marked dirty and refreshed before the next step. Tiles with $E_t \le s$ are refreshed during selection, and the salt changes only in a perturbation, after which all tiles are rebuilt. So by Lemma 1 every summary is correct when it is read. For one tile, the best allowed vertex is either the best non-tabu vertex F_t, or a tabu vertex that passes the aspiration test. The aspiration test cut + g > best is monotone in g, and a larger key always has a larger or equal gain. So if any tabu vertex of the tile passes the test, the one with the largest key T_t passes it too and is the best tabu candidate. Taking the larger of the two candidates for every tile and then the maximum over all tiles gives the maximum key over $A(s)$. ∎

Because the keys are unique (the low half is a bijection of the vertex id), there are no ties, and the result does not depend on the order in which the threads look at the tiles. We also check Theorem 1 experimentally in Section VI-A.

Cost. Let $D(v)$ be the number of different tiles in $N(v) \cup \{v\}$ and X_t the number of expired tiles in a step. The entries read per step are

$$W_{\text{step}} = \lceil n/32 \rceil + 32\,D(v) + 32\,X_t \qquad (5)$$

instead of n. Since $D(v) \le \deg(v)+1$, the selection is never worse than a full scan by more than a constant, and for sparse graphs it is much smaller. For dense graphs where $D(v)$ is close to $n/32$ there is no gain, which we also see in the results. Table III summarises the cost of one step for the main versions.

TABLE III
COST OF ONE STEP FOR ONE SEARCH (P THREADS PER BLOCK)

	Naive	DA	DATS
selection reads	n	n	$n/32 + 32D(v)$
selection time	n/P	n/P	$n/(32P) + D(v)\cdot 32/P$
update time	$\deg(v)$	$\approx\deg(v)/32$ or $/P$	$\approx\deg(v)/32$ or $/P$
extra state	–	–	20 B per tile
exact BLS moves	yes	yes	yes

C. Degree-Aware Neighbour Update

For the update in (3), the vertices are divided into three classes on the host. If $\deg(v) \le 32$, warp 0 updates all neighbours in one go, one per lane. If $32 < \deg(v) \le 1024$, warp 0 loops over the list 32 neighbours at a time. Above 1024, all warps of the block share the list. Every neighbour appears once in the CSR row, so no two threads write the same gain and no atomics are needed for the gains. The low and medium classes use only one warp so that the other warps are not involved for short lists, which keeps the update simple. Tuning these limits is left for future work.

D. BFS Relabelling

The number of dirty tiles depends on how the neighbours of v are spread over the ids. We relabel the vertices with a breadth-first search that starts from the vertex of highest degree, so the vertices found from the same parent get consecutive ids. Unreached components start from the next highest degree vertex. The relabelling is done once on the host in $O(n + m)$ time.

E. GPU Implementation

The shared memory of a block holds, in this order, the tile summaries (20 bytes per tile), the dirty bitmask, the solution (1 bit per vertex) and the gains (4 bytes per vertex) if they still fit, otherwise the gains stay in a global memory slice of the block. The tile structure costs only 0.625 bytes per vertex, so with 64 KB of shared memory graphs with about 80k vertices keep everything except the gains on chip. The tabu array is kept in global memory. Thread 0 owns the random number generator, the current and best cut and the perturbation state. The complete kernel step is given in Algorithm 1. All our versions use the same xorshift generator [28] and seeds, so block i on the GPU and search i on the CPU are designed to make exactly the same moves.

Fig. 6 shows how much shared memory a block needs on a T4 for some of our graphs. For small graphs like G55 everything fits, including the gains (23.2 KB). For email-Enron the gains alone would need 143 KB, so they stay in global memory, and only the tiles, bitmask and solution are on chip (27.0 KB). Even soc-Epinions1 with 76k vertices needs only 55.9 KB for these parts, which leaves room for two blocks per SM.

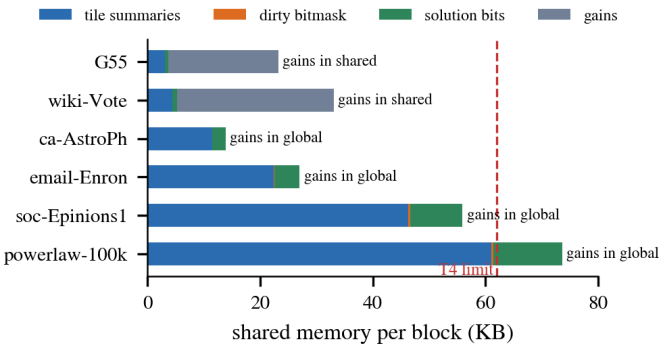


Fig. 6. Shared memory used per block on the T4 (64 KB limit, minus 2 KB we keep for static variables). Gains are kept in shared memory only when everything fits.

Algorithm 1: DATS kernel (one block)

```
for each block b in parallel:            // one search per block
  x <- random bits(seed_b); compute gains g
  s <- rand(); build all tile summaries
  for step = 1 .. T:
    for each tile t (strided over threads):
      if E[t] <= step: refresh(t)
      c <- F[t]
      if T[t] aspirates and T[t] > c: c <- T[t]
    v <- vertex of block_max(c)       // shuffle
    if deg(v) <= 32:   warp 0, 1 neighbour/lane
    elif deg <= 1024:  warp 0 loops over N(v)
    else:              all warps share N(v)
       (update g[u], atomicOr dirty[tile(u)])
    flip x[v]; g[v] <- -g[v]; make v tabu
    each warp: refresh its dirty tiles
    if stuck: random flips, s <- rand(),
              rebuild all tiles
  write best cut and solution of block b
```

Algorithm 2 shows the tile refresh in more detail. The dirty bitmask has one bit per tile, so it needs only $n/256$ bytes. Each warp takes some 32-bit words of the mask and uses `__ffs` (find first set bit) to jump directly to the dirty tiles, so clean tiles cost nothing. Inside a tile, lane i handles vertex $32t+i$, and three shuffle reductions of five steps each give the two maxima and the minimum expiry. No shared memory atomics are needed here, because each tile is refreshed by exactly one warp.

Algorithm 2: refreshing the dirty tiles

```
refresh_dirty(next_step):              // all warps
  for w = warp_id; w < n_words; w += n_warps:
    bits <- dirty[w]
    while bits != 0:
      b <- ffs(bits) - 1; bits <- bits & (bits-1)
      t <- 32*w + b;  v <- 32*t + lane
      k <- key(g[v], v, s)
      if tabu_until[v] > next_step:
           free <- 0;  tabu <- k;  exp <- tabu_until[v]
      else free <- k;  tabu <- 0;  exp <- INF
      for off in 16, 8, 4, 2, 1:           // shuffles
        free <- max(free, shfl_down(free, off))
        tabu <- max(tabu, shfl_down(tabu, off))
        exp  <- min(exp,  shfl_down(exp,  off))
      if lane == 0: F[t], T[t], E[t] <- free, tabu, exp
    if lane == 0: dirty[w] <- 0
```

V. Experimental Setup

Hardware. The CPU experiments were run on an Apple M5 machine with 10 cores. The GPU experiments were run on an NVIDIA Tesla T4 (40 SMs, 16 GB) on Google Colab. All code is C++17 with OpenMP and CUDA, compiled with -O3, and is in a single source file.

Graphs. Table II lists the graphs. From the G-set [7] we use four random graphs (G1, G22, G43, G55), a planar graph (G14) and a toroidal grid (G48). From SNAP [14] we use wiki-Vote, email-Enron, ca-AstroPh and soc-Epinions1 as undirected, unweighted graphs. We also generate a Chung–Lu power-law graph [15] with 100k vertices, exponent 2.1 and average degree 8, which has much bigger hubs. Fig. 7 shows the degree distributions.

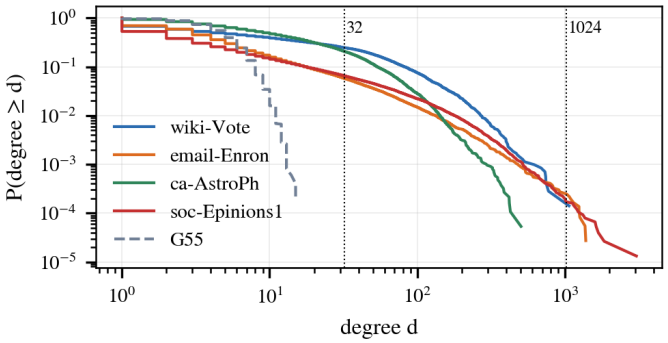


Fig. 7. Degree distribution (complementary CDF) of the SNAP graphs compared with the G-set graph G55. Dotted lines are the class limits 32 and 1024.

Parameters. Unless stated, each version runs 8 searches of 10,000 steps. The tabu tenure is 3 + rand(0, n/10), perturbation starts after 500 non-improving steps, and its strength goes from max(2, n/100) up to n/20, capped at 100 and 500. On the GPU we use 256 threads per block and two blocks per SM.

Versions. On the CPU: *full* (serial full scan), *tiled* (serial, tiled selection), *tiled+BFS*, and the OpenMP versions of full and tiled+BFS, which run the searches in parallel. On the GPU: *Naive* (full scan, one thread per vertex update, as in [5]), *DA* (full scan, degree-aware update), *DA+T* (degree-aware + tiles), *T+R* (tiles + relabel with naive update) and *DATS* (all three). We report moves per second (including perturbation moves), the work of the selection (entries read per step), dirty tiles per step and the best cut. Every best cut reported by a version is recomputed from its solution with an independent function.

Implementation and validation. All versions are in one C++/CUDA source file of about 1,600 lines. The CPU versions use OpenMP [27] with a dynamic schedule, one search per iteration. The GPU part is compiled only when the file is built with nvcc, so the same file also builds as a CPU-only program. Because all versions use the same xorshift generator [28] and the same seeds, and the tiled selection is exact (Theorem 1), a version can be checked against another one by comparing the best cut of every search, not just the overall best. The program prints these checks after every run, and a run that fails them is treated as a bug. All tables and figures in this paper are generated by a script directly from the CSV files written by the program, so no number is copied by hand. The appendix lists the exact commands.

Timing. CPU times are wall-clock times measured with `std::chrono` around all searches. GPU times are measured with CUDA events around the kernel only, after one untimed warm-up launch, and do not include copying the graph (which is done once). Because many G-set runs take only a few milliseconds, small differences on those graphs should not be over-interpreted.

VI. Results

A. Correctness

For all 15 graph inputs (including the shuffled ones), the serial tiled version found the same best cut as the full scan for every one of the 8 searches, which confirms the exactness argument of Section IV. All reported cuts matched the recomputed value. On the GPU, all variants that use the same vertex order found the same best cut on every graph, and all GPU cuts matched the recomputed value.

B. Selection Work

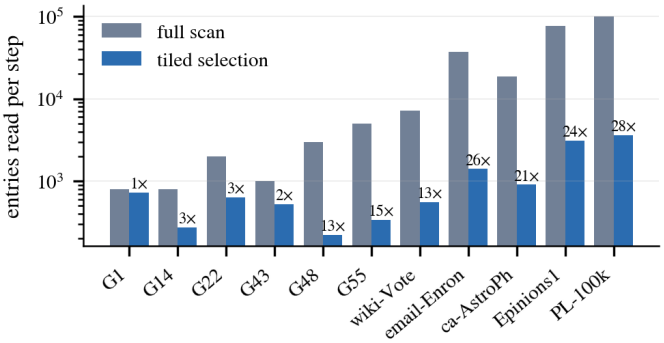


Fig. 8. Entries read per step to choose a move (log scale), full scan versus tiled selection. The number on top is the reduction.

Fig. 8 shows the average number of entries read per step. On the sparse graphs the tiled selection reads 13–28× fewer entries. For email-Enron, for example, it reads about 1,405 entries instead of 36,692. Most of what is left is the scan over the n/32 summaries. On the dense random graphs G1 and G43 a flip makes 21.5 of 25 tiles and 14.8 of 32 tiles dirty, so the reduction is less than 2×.

C. Where the Selection Work Goes

Equation (5) splits the work of the tiled selection into three parts: the scan over the n/32 summaries, the refresh of dirty tiles, and the refresh of expired tiles plus the full rebuilds after each perturbation. Fig. 9 shows the measured split. On the large graphs the summary scan is 65–87% of the work and the dirty refresh 13–32%. On the smaller sparse graphs like G48 the dirty refresh is a larger part (44%). This tells us where to look next: on large graphs a second level of summaries would remove most of the remaining work, while on small graphs the dirty refresh and the per-perturbation rebuilds matter more.

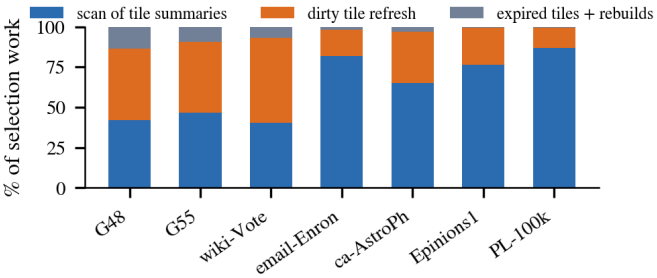


Fig. 9. Split of the tiled selection work into the scan of tile summaries, the refresh of dirty tiles, and expired tiles plus rebuilds after perturbations (serial tiled CPU version).

D. CPU Speedup

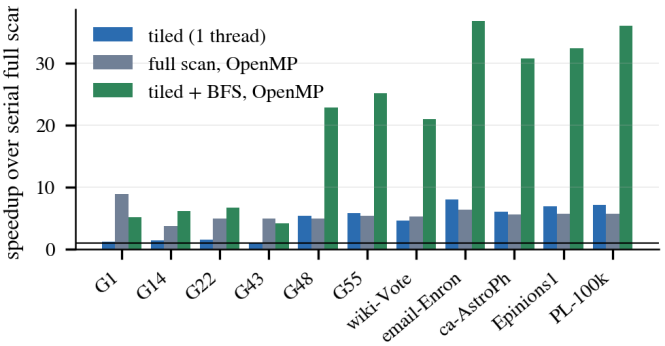


Fig. 10. Speedup in moves per second over the serial full-scan search (8 searches, 10,000 steps each, 10 OpenMP threads).

Fig. 10 shows that the lower selection work also gives real time savings. On one core, tiled selection is 4.6–8.0× faster on the sparse graphs. Running the searches in parallel with OpenMP gives 4.9–6.3× for the full scan, and the two together give 20.9–36.8×, i.e. the gains multiply. On G1 and G43 there is no gain from tiling, and on G1 the OpenMP tiled version was even a bit slower (5.1× vs 8.9×) because of the extra bookkeeping. Note that these G-set runs take only a few milliseconds, so small differences there are within timing noise.

E. Scaling with OpenMP Threads

Fig. 11 shows how the OpenMP versions scale on email-Enron and soc-Epinions1 when 10 searches are shared among 1 to 10 threads. At 10 threads the full scan reaches 6.0× and 6.4×, and the tiled version 5.3× and 5.0×. The searches are independent, so the scaling is limited mainly by the hardware and by the uneven split of 10 searches: with 6 or 8 threads the searches still need two rounds, which is why these two points give almost the same speed. The tiled version scales a little less than the full scan. We think this is because each of its steps is much shorter and does more scattered memory accesses, so the memory system becomes the limit earlier, but we did not measure this. In absolute terms it is still much faster at every thread count, e.g. 7.1× with 1 thread and 6.2× with 10 threads on email-Enron.

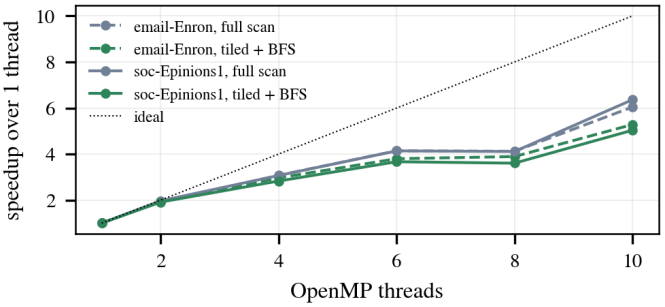


Fig. 11. OpenMP strong scaling (10 searches of 10,000 steps) on email-Enron (dashed) and soc-Epinions1 (solid). The dotted line is ideal scaling.

F. Scaling with Graph Size

To see how the method behaves as graphs grow, we generated Chung–Lu graphs with average degree 8 and 5,000 to 200,000 vertices (the largest hub grows from 1,329 to 30,749 neighbours). Fig. 12(a) shows the time per move of the serial versions. The full scan grows linearly with n, as expected. The tiled versions also grow, because of the $n/32$ summary scan, but they stay 6.1–7.9× faster, and the reduction in selection work grows from 16× to 28×. Fig. 12(b) shows that the number of dirty tiles per step also grows with n, because the hubs get bigger. Here BFS relabelling helps more for larger graphs: at 200,000 vertices it reduces the dirty tiles from 24.9 to 19.3 per step (23%), while at 5,000 the difference is only 2%.

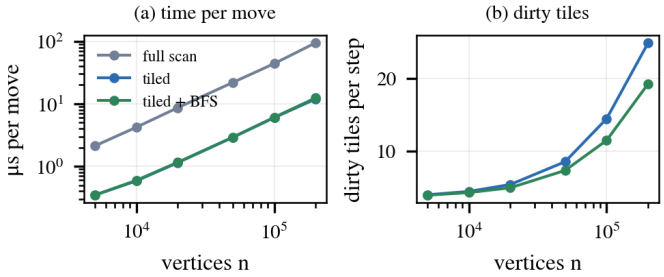


Fig. 12. Scaling with graph size on Chung–Lu power-law graphs (average degree 8, serial CPU, 4 searches). (a) Time per move. (b) Dirty tiles per step for the tiled versions.

G. Choice of Tile Size

On the GPU the tile size is fixed to 32 by the warp size, but on the CPU we can change it and check whether 32 is a good choice. With tile size T, the work per step is roughly

$$W(T) \approx \lceil n/T \rceil + T \cdot D_T(v) \qquad (6)$$

where $D_T(v)$ is the number of size-T tiles touched by a flip. Small tiles make the summary scan long, and large tiles make every dirty tile expensive, so there should be a best value in between. Fig. 13 confirms this. The selection work and the run time are both U-shaped in T. The fastest tile size was 32 for wiki-Vote, 128 for email-Enron, 64 for soc-Epinions1, 32 for G55. So 32 is the best choice for the smaller graphs, while the large graphs prefer larger tiles: on email-Enron the best size is 1.49× faster than 32, soc-Epinions1 the best size is 1.39× faster than 32. This matches the work breakdown of Fig. 9, where the summary scan dominates on large graphs. On the GPU we keep 32 because one warp refreshes one tile, but a second level of summaries, or two warps per tile, could give the same benefit there.

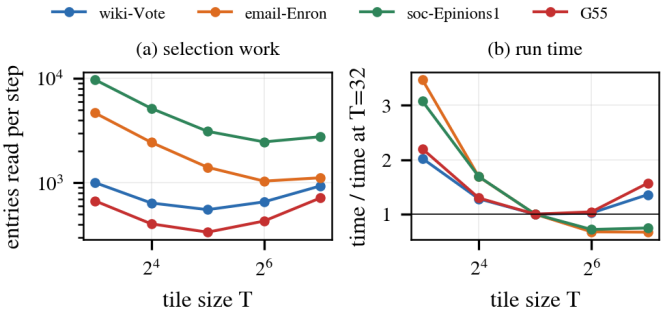


Fig. 13. Effect of the tile size on the serial tiled CPU search. (a) Entries read per step. (b) Run time relative to $T = 32$.

H. Effect of Relabelling

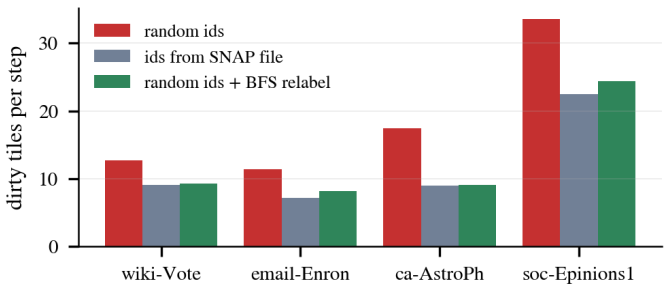


Fig. 14. Dirty tiles per step for the tiled CPU search when vertex ids are as in the SNAP file, randomly shuffled, and shuffled then BFS relabelled.

The SNAP files already have a fairly good order, because ids were given while crawling, so BFS relabelling changes little there. To test the case where the ids are arbitrary, we randomly shuffled the ids before running. As Fig. 14 shows, shuffling increases the dirty tiles a lot, and BFS relabelling brings them back close to the level of the original order, a reduction of 26–48% compared with the shuffled order. It does not beat the crawl order of the SNAP files on every graph, so a better relabelling (for example community based) could still help. On the G-set graphs BFS relabelling changed the dirty tiles by at most 10%.

I. GPU Performance

TABLE IV
GPU THROUGHPUT IN MOVES PER SECOND (TESLA T4, 80 BLOCKS)

Graph	Naive	DA	DA+T	T+R	DATS
G1	8.47M	26.34M	7.32M	6.02M	9.58M
G14	38.91M	55.03M	19.35M	17.23M	20.32M
G22	21.84M	38.45M	13.32M	10.50M	13.97M
G43	25.69M	50.54M	12.22M	9.79M	12.64M
G48	27.25M	27.73M	21.28M	21.11M	23.10M
G55	14.75M	14.45M	22.20M	21.78M	24.04M
wiki-Vote	10.02M	11.26M	10.38M	7.04M	10.53M
email-Enron	702k	699k	11.04M	8.50M	11.20M
ca-AstroPh	1.36M	1.41M	11.71M	8.38M	12.33M
soc-Epinions1	325k	332k	3.83M	2.32M	3.88M
powerlaw-100k	254k	259k	2.33M	2.13M	2.38M

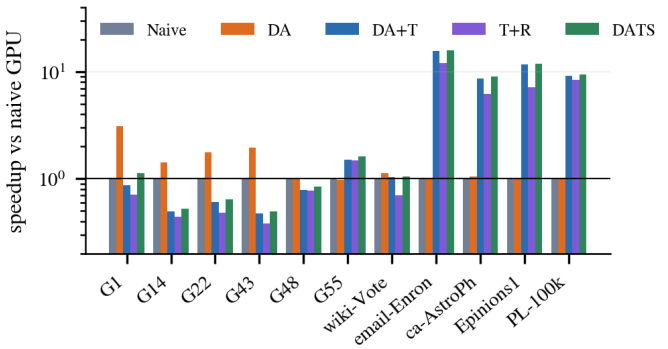


Fig. 15. Speedup of the GPU variants over the naive GPU version (log scale). Naive: full scan, one thread updates; DA: full scan, degree-aware update; DA+T: with tiles; T+R: tiles and relabelling with naive update; DATS: all three.

Table IV and Fig. 15 give the GPU results. The behaviour is very different for large and small graphs, so we discuss them separately.

Large graphs. On the four graphs with more than 15k vertices, degree-aware updates with the full scan change the speed by only 1.00–1.04×, which is what the cost analysis of Section III predicts: the scan dominates and balancing the update does not matter. Tiling removes this bottleneck, and DATS is 9.1–15.9× faster than the naive version. Once tiling is used, the degree-aware update itself gives 1.09–1.68× (DATS versus T+R, over all graphs), because the update is now a much bigger part of a step. So the two techniques are useful together, but degree-aware updates alone are not.

Small graphs. On the G-set graphs with at most 2000 vertices the picture is reversed. Here the full scan reads only 3–7 entries per thread, which is very cheap, and DA is already 1.4–3.1× faster than naive. The tile refresh adds extra synchronisation and atomic operations to every step, so DATS reaches only 0.25–0.37× of the DA speed. The crossover is somewhere between a few thousand and ten thousand vertices: on G48 (3000 vertices) DATS gets 0.83× of DA, on wiki-Vote (7115) 0.94×, and on G55 (5000, very sparse) 1.66×. For small dense graphs a full scan is the better choice.

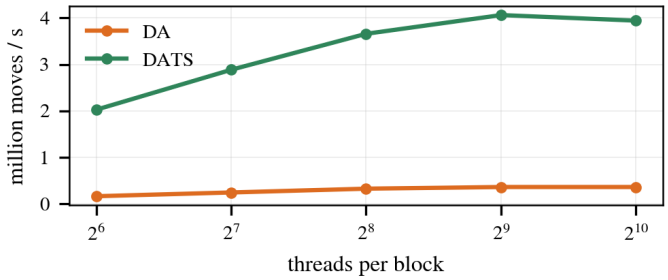


Fig. 16. Throughput on soc-Epinions1 for different block sizes.

Fig. 16 shows the effect of the block size on soc-Epinions1. DATS is fastest with 512 threads per block. We think larger blocks help the hub updates and the tile scan but make every __syncthreads more expensive.

TABLE V
EFFECT OF BFS RELABELLING ON THE GPU (MOVES/S)

Graph	file ids: DA+T	file ids: DATS	random ids: DA+T	random ids: DATS
wiki-Vote	10.38M	10.53M	10.99M	10.67M
email-Enron	11.04M	11.20M	9.48M	10.63M
ca-AstroPh	11.71M	12.33M	9.21M	12.05M
soc-Epinions1	3.83M	3.88M	3.18M	3.84M

Table V repeats the relabelling study on the GPU. DA+T and DATS differ only in the BFS relabelling. With the ids from the SNAP files, relabelling changes the speed by 1.01–1.05×, so it neither helps nor hurts much. With random ids it gives 0.97–1.31×, and DATS on random ids is almost as fast as on the original ids. So on the GPU, too, relabelling mainly protects against a bad input order.

J. CPU versus GPU

TABLE VI
CPU VERSUS GPU THROUGHPUT (MILLION MOVES/S)

Graph	CPU 1 core	CPU best (10 cores)	GPU best	GPU / CPU best	GPU / 1 core
G1	1.26	11.17	26.34 (DA)	2.4×	21×
G14	2.82	17.24*	55.03 (DA)	3.2×	20×
G22	1.17	7.87*	38.45 (DA)	4.9×	33×
G43	2.22	10.86	50.54 (DA)	4.7×	23×
G48	0.80	18.25*	27.73 (DA)	1.5×	35×
G55	0.48	12.07*	24.04 (DATS)	2.0×	50×
wiki-Vote	0.35	7.40*	11.26 (DA)	1.5×	32×
email-Enron	0.05	2.01*	11.20 (DATS)	5.6×	205×
ca-AstroPh	0.12	3.70*	12.33 (DATS)	3.3×	102×
soc-Epinions1	0.03	0.97*	3.88 (DATS)	4.0×	130×
powerlaw-100k	0.02	0.82*	2.38 (DATS)	2.9×	105×

Table VI compares the CPU and the GPU directly. For each graph we take the fastest CPU version (full scan or tiled + BFS, both with OpenMP; * marks the tiled one) and the fastest GPU version. The GPU is faster than the best CPU version on all 11 graphs. On the large graphs it is 2.9–5.6× faster than all 10 CPU cores and 102–205× faster than one core. On the smaller graphs the gap is 1.5–4.9×.

The table also shows that the best method depends on the graph size on both devices. On the large graphs the naive GPU version is only 1.9–2.0× faster than the OpenMP full scan, because both spend most of their time scanning all vertices. Tiling is what opens the gap, and it helps the CPU as well: the tiled CPU version is the fastest CPU version on 9 of 11 graphs. On the GPU, the simple degree-aware full scan (DA) is the fastest version on 6 of the 7 graphs with at most 7,115 vertices, and DATS is the fastest on all large graphs.

In solution quality the GPU found an equal or better cut than the CPU on every graph. This comes from running 80 searches instead of 8, not from a better search, since one search makes the same moves on both devices. Also, the CPU (Apple M5) and the GPU (Tesla T4) are very different machines, so these ratios are only a rough comparison. The comparisons inside one device (Figs. 10 and 15) are the main results of this paper.

K. Solution Quality

TABLE VII
BEST CUT ON G-SET (10 SEARCHES × 100,000 STEPS)

Graph	Best known [4]	Full (OMP)	DATS (OMP)	Gap
G1	11,624	11,624	11,624	0.00%
G14	3,064	3,056	3,056	0.26%
G22	13,359	13,351	13,353	0.04%
G43	6,660	6,659	6,659	0.02%
G48	6,000	6,000	6,000	0.00%
G55	10,294	10,116	10,117	1.72%

Since DATS does not change the search itself, its quality is the same as our BLS-style tabu search. Table VII compares the best cut of 10 searches of 100,000 steps with the best known values reported in [4]. We reach the best known value on G1 and G48. On G14, G22, G43 we are within 0.26%. On the larger G55 the gap is bigger, since BLS uses much longer runs and some extra rules (directed perturbation) that we did not implement. The full and DATS versions differ slightly only because the BFS relabelling changes the starting solutions.

Table VII compares the two versions after the same number of steps. In practice, what matters is the quality after the same *time*. Fig. 17 shows the best cut against wall time for runs of 1,000 to 100,000 steps (10 searches, OpenMP). Since the tiled version does each step faster, its curve is shifted to the left: on G22, in 0.10 s the tiled version finishes 100,000 steps and finds 13,353, while in the same time the full scan only gets through 30,000 steps and finds 13,304; on G55, in 0.06 s the tiled version finishes 100,000 steps and finds 10,117, while in the same time the full scan only gets through 10,000 steps and finds 9,928; on email-Enron, in 0.32 s the tiled version finishes 100,000 steps and finds 118,107, while in the same time the full scan only gets through 10,000 steps and finds 114,798. For a fixed time budget, a faster step directly gives a better cut.

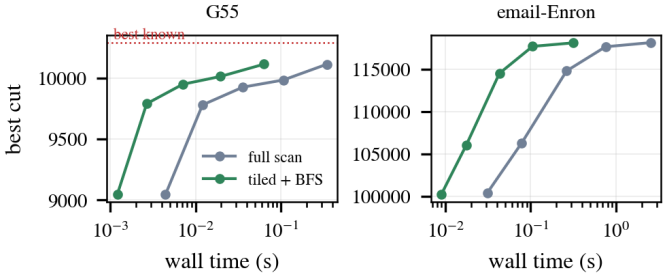


Fig. 17. Best cut against wall time for runs of 1,000 to 100,000 steps (10 searches with OpenMP). The dotted line is the best known value from [4].

VII. Discussion

A. Limitations

Our results show that for local search on sparse power-law graphs the cost of choosing the move matters more than load balancing of the update, and that tiling and degree-aware updates work best together. There are some limitations. (1) On dense graphs a single flip makes most tiles dirty, so tiling does not help, and on small graphs the GPU full scan is already so cheap that tiling makes it slower. A simple rule based on n and the average $D(v)$ could choose between the two methods. (2) The top level still scans $n/32$ summaries, which is now the largest part of the work on our biggest graphs. A second level of summaries (tiles of tiles) would make it about $n/1024$. (3) The tile summaries must fit in shared memory for the best speed, which limits n to about 10^5 on current GPUs. (4) We use one search per block; for small graphs a warp per search could use the GPU better. (5) Hubs are flipped in very few steps, so the benefit of the degree-aware update is limited by how often hubs are chosen, which depends on the search rules. A different search, for example one that flips groups of vertices, could make the update more important.

B. Threats to Validity

Hardware. The CPU and GPU results come from two different machines (an Apple M5 CPU and a data-centre GPU on Colab), so the CPU versus GPU ratios depend on these machines. Comparisons inside one device are not affected by this. Colab GPUs are shared, and we ran each configuration once, so small differences between GPU variants (a few percent) may be noise. **Graphs.** We used six G-set graphs, four real SNAP graphs and synthetic power-law graphs. Other graph families, such as road networks with very low degree or very dense graphs, may behave differently, although the cost model of Eq. (5) predicts the trend. **Search.** We implemented the main BLS rules but not all of them, and we used fixed parameters on all graphs. Our quality numbers are therefore lower than the best published ones on hard instances. This does not affect the speed comparisons, because all versions make the same moves. **Metric.** Moves per second counts perturbation moves as well, which are cheaper than normal moves on the GPU. All versions do the same number of perturbation moves, so the comparison stays fair.

C. Which Method Should Be Used?

From the results we can give a simple rule. For small or dense graphs (a few thousand vertices, or $D(v)$ close to $n/32$), a full scan with degree-aware updates is the best on the GPU, and the plain OpenMP full scan is enough on the CPU. For large sparse graphs, especially power-law graphs with hubs, the full DATS combination is clearly better on both devices. If the vertex ids come in an unknown or random order, BFS relabelling should be applied, since it is cheap and never hurt much in our tests. Since n, the average degree and an estimate of $D(v)$ are known before the search starts, this choice could easily be made automatically by the program.

VIII. Conclusion

We presented DATS, a GPU tabu search for Max-Cut that combines exact tiled incremental selection, degree-aware neighbour updates and BFS relabelling. The idea came from noticing that the full scan for the best move, and not the hub updates, is the main cost of one step on sparse graphs. The tiled selection gives exactly the same moves as a full scan while reading 13–28× fewer entries, and with OpenMP on a CPU the combined speedup reaches 37×. On a Tesla T4 GPU, DATS is 9.1–15.9× faster than a one-thread-per-vertex design on large sparse graphs, while on small dense graphs a plain full scan remains better. We also showed that the best method depends on the graph: for small dense graphs a full scan is better, while for large sparse graphs the tiled selection wins by a large factor.

In future we want to (1) add a second level of summaries so that the remaining $n/32$ scan becomes about $n/1024$, (2) choose between full scan and tiling automatically from the graph, (3) run several searches per block, one per warp, for small graphs, (4) try better orderings such as Rabbit Order [23], and (5) apply the same tiling to other problems that are solved with flip-based local search, such as general QUBO, graph partitioning and MAX-SAT.

Appendix: Reproducing the Results

All code, scripts and data needed to reproduce this paper are in one folder. The graphs are downloaded automatically from the G-set and SNAP websites. The steps are:

```
# CPU experiments (any Linux or macOS machine)
./run_experiments.sh cpu
./run_experiments.sh extra
# GPU experiments (CUDA machine, e.g. Colab T4)
./run_experiments.sh gpu
# build this paper from results/*.csv
pip install reportlab==4.0.9 matplotlib numpy
python3 paper/make_paper.py
```

The single source file `maxcut_research.cpp` builds with `nvcc -O3 -arch=sm_75 -x cu -Xcompiler -fopenmp` for the GPU, or with `g++ -O3 -fopenmp` for the CPU only. Table VIII lists all search parameters. Every run prints whether the tiled and full versions, and all GPU variants, gave the same results.

TABLE VIII
SEARCH PARAMETERS USED IN ALL EXPERIMENTS

Parameter	Value
steps per search	10,000 (quality: 100,000)
CPU searches	8 (quality and scaling: 10)
GPU blocks / threads	80 / 256
tabu tenure	$3 + \text{rand}(0, n/10)$
stall limit before perturbation	500 steps
perturbation strength	$\max(2, n/100)$ to $n/20$ (capped 100, 500)
degree classes	≤ 32, 33–1024, > 1024
tile size	32 (CPU ablation: 8–128)
random generator / seed	xorshift32 / 2024

Acknowledgment

We thank our Parallel Computing course faculty at VJTI for their guidance during this project.

References

[1] R. M. Karp, "Reducibility among combinatorial problems," in *Complexity of Computer Computations*, R. E. Miller and J. W. Thatcher, Eds. New York: Plenum, 1972, pp. 85–103.

[2] A. Lucas, "Ising formulations of many NP problems," *Frontiers in Physics*, vol. 2, art. 5, 2014.

[3] M. X. Goemans and D. P. Williamson, "Improved approximation algorithms for maximum cut and satisfiability problems using semidefinite programming," *J. ACM*, vol. 42, no. 6, pp. 1115–1145, 1995.

[4] U. Benlic and J.-K. Hao, "Breakout local search for the max-cut problem," *Engineering Applications of Artificial Intelligence*, vol. 26, no. 3, pp. 1162–1173, 2013.

[5] C. Cook, H. Zhao, T. Sato, M. Hiromoto, and S. X.-D. Tan, "GPU-based Ising computing for solving max-cut combinatorial optimization problems," *Integration, the VLSI Journal*, Elsevier, 2019.

[6] R. Yasudo, K. Nakano, Y. Ito, M. Tatekawa, R. Katsuki, T. Yazane, and Y. Inaba, "Adaptive bulk search: Solving quadratic unconstrained binary optimization problems on multiple GPUs," in *Proc. 49th Int. Conf. Parallel Processing (ICPP)*, ACM, 2020.

[7] C. Helmberg and F. Rendl, "A spectral bundle method for semidefinite programming," *SIAM J. Optimization*, vol. 10, no. 3, pp. 673–696, 2000.

[8] F. Glover, "Tabu search—Part I," *ORSA J. Computing*, vol. 1, no. 3, pp. 190–206, 1989.

[9] R. Yasudo, K. Nakano, Y. Ito et al., "Graph-theoretic formulation of QUBO for scalable local search on GPUs," in *Proc. IEEE Int. Parallel and Distributed Processing Symp. Workshops (IPDPSW)*, 2022, pp. 425–434.

[10] D. Merrill, M. Garland, and A. Grimshaw, "Scalable GPU graph traversal," in *Proc. 17th ACM SIGPLAN Symp. Principles and Practice of Parallel Programming (PPoPP)*, 2012, pp. 117–128.

[11] Y. Wang, A. Davidson, Y. Pan, Y. Wu, A. Riffel, and J. D. Owens, "Gunrock: A high-performance graph processing library on the GPU," in

Proc. 21st ACM SIGPLAN Symp. PPoPP, 2016.

[12] C. M. Fiduccia and R. M. Mattheyses, "A linear-time heuristic for improving network partitions," in *Proc. 19th Design Automation Conf.*, 1982, pp. 175–181.

[13] E. Cuthill and J. McKee, "Reducing the bandwidth of sparse symmetric matrices," in *Proc. 24th ACM National Conf.*, 1969, pp. 157–172.

[14] J. Leskovec and A. Krevl, "SNAP Datasets: Stanford large network dataset collection," 2014. [Online]. Available: http://snap.stanford.edu/data

[15] F. Chung and L. Lu, "The average distances in random graphs with given expected degrees," *Proc. National Academy of Sciences*, vol. 99, no. 25, pp. 15879–15882, 2002.

[16] S. Burer, R. D. C. Monteiro, and Y. Zhang, "Rank-two relaxation heuristics for MAX-CUT and other binary quadratic programs," *SIAM J. Optimization*, vol. 12, no. 2, pp. 503–521, 2002.

[17] G. A. Kochenberger, J.-K. Hao, Z. Lü, H. Wang, and F. Glover, "Solving large scale max cut problems via tabu search," *J. Heuristics*, vol. 19, no. 4, pp. 565–571, 2013.

[18] T. Inagaki et al., "A coherent Ising machine for 2000-node optimization problems," *Science*, vol. 354, no. 6312, pp. 603–606, 2016.

[19] H. Goto, K. Tatsumura, and A. R. Dixon, "Combinatorial optimization by simulating adiabatic bifurcations in nonlinear Hamiltonian systems," *Science Advances*, vol. 5, no. 4, eaav2372, 2019.

[20] M. Yamaoka et al., "A 20k-spin Ising chip to solve combinatorial optimization problems with CMOS annealing," *IEEE J. Solid-State Circuits*, vol. 51, no. 1, pp. 303–309, 2016.

[21] S. Hong, S. K. Kim, T. Oguntebi, and K. Olukotun, "Accelerating CUDA graph algorithms at maximum warp," in *Proc. 16th ACM Symp. PPoPP*, 2011, pp. 267–276.

[22] H. Wei, J. X. Yu, C. Lu, and X. Lin, "Speedup graph processing by graph ordering," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, 2016, pp. 1813–1828.

[23] J. Arai, H. Shiokawa, T. Yamamuro, M. Onizuka, and S. Iwamura, "Rabbit Order: Just-in-time parallel reordering for fast graph analysis," in *Proc. IEEE IPDPS*, 2016, pp. 22–31.

[24] A.-L. Barabási and R. Albert, "Emergence of scaling in random networks," *Science*, vol. 286, no. 5439, pp. 509–512, 1999.

[25] NVIDIA, "CUDA C++ Programming Guide." [Online]. Available: https://docs.nvidia.com/cuda/cuda-c-programming-guide/

[26] Q. Wu and J.-K. Hao, "A memetic approach for the max-cut problem," in *Parallel Problem Solving from Nature (PPSN XII)*, LNCS 7492, Springer, 2012, pp. 297–306.

[27] L. Dagum and R. Menon, "OpenMP: An industry standard API for shared-memory programming," *IEEE Computational Science and Engineering*, vol. 5, no. 1, pp. 46–55, 1998.

[28] G. Marsaglia, "Xorshift RNGs," *J. Statistical Software*, vol. 8, no. 14, pp. 1–6, 2003.

[29] M. Harris, "Optimizing parallel reduction in CUDA," NVIDIA Developer Technology, 2007.